

6-bit Processor with Assembler

Prepared by:

Masoumeh Ostvar

Department of Computer Engineering

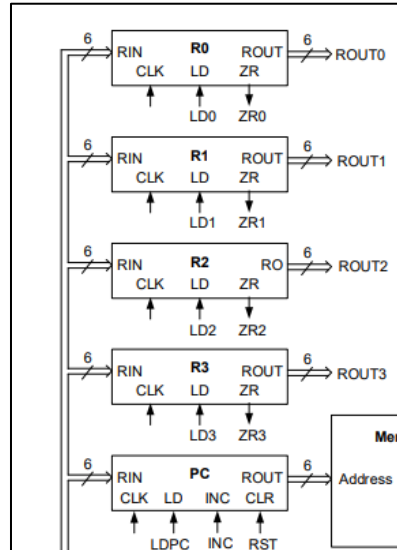
Spring 2024

Contents

Step 1: Basic Processor	2
Implementation of Step 1	7
Step 2: Software Multiplication	8
Implementation of Step 2	12
Step 3: Hardware Multiplication Extension	13
Implementation of Step 3	14
Python Assembler.....	14

Step 1: Basic Processor

To implement this circuit, we first need registers, which are required for implementing both the FSM and the processor registers.



For this purpose, we first define a type for the different FSM states and declare two signals, CurrentState and NextState. We then define the remaining processor registers.

```
13 type State_t is (S0, HaltCheck, S1, S2, S3, S4, S5, S6, S7);
```

```
42 --FSM
43 signal CurrentState, NextState : State_t;
44 -- Registers
45 signal R0,R1,R2,R3,IR,PC : std_logic_vector (5 downto 0);
46 signal R0Next,R1Next,R2Next,R3Next,IRNext,PCNext : std_logic_vector (5 downto 0);
```

The registers were implemented using a process sensitive to the clock edge and reset signals:

```
64 Registers: process(clk,reset)
65 begin
66     if reset='1' then
67         CurrentState <= s0;
68         IR <= (others => '0');
69         PC <= (others => '0');
70         R0 <= (others => '0');
71         R1 <= (others => '0');
72         R2 <= (others => '0');
73         R3 <= (others => '0');
74     elsif (rising_edge(clk)) then
75         CurrentState <= NextState;
76         IR <= IRNext;
77         PC <= PCNext;
78         R0 <= R0Next;
79         R1 <= R1Next;
80         R2 <= R2Next;
81         R3 <= R3Next;
82     end if;
83 end process;
```

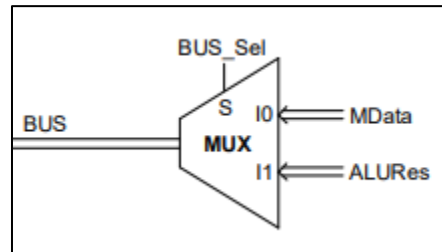
The remaining signals in the processor architecture were defined as follows:

```

47 -- Controls
48 signal MData, DataBUS, ALURes, IN1, IN2 : std_logic_vector(5 downto 0);
49 signal SelMux1, SelMux2 : std_logic_vector(1 downto 0);
50 signal ZR0, ZR1, ZR2, ZR3, BUSSel, LDPC, LDIR, INC, RST, CMD, LD0, LD1, LD2, LD3 : std_logic;

```

According to the project specification, the multiplexer in the processor architecture is implemented as follows:



The sensitivity list was defined according to the figure above, and its functionality was implemented as follows:

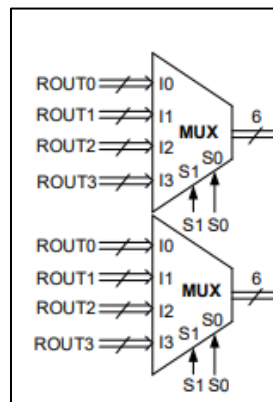
```

86 Mux0: process(MData, ALURes, BUSSel)
87 begin
88     case BUSSel is
89         when '0' =>
90             DataBUS <= MData;
91         when '1' =>
92             DataBUS <= ALURes;
93         when others =>
94             DataBUS <= (others => '0');
95     end case;
96 end process;

```

As shown, when the MUX select signal is set to 0, MData is passed to the output. When it is set to 1, ALURes is passed to the output.

According to the project specification, the other two multiplexers in the processor architecture are implemented as follows:



The sensitivity list was defined according to the figure above, and the functionality of each multiplexer was implemented as follows:

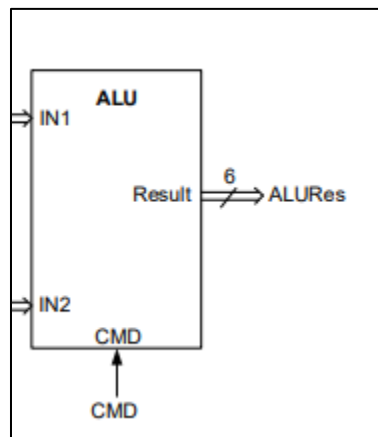
```

98   Mux1: process(R0,R1,R2,R3,SelMux1)
99   begin
100     case SelMux1 is
101     when "00" =>
102       IN1 <= R0;
103     when "01" =>
104       IN1 <= R1;
105     when "10" =>
106       IN1 <= R2;
107     when "11" =>
108       IN1 <= R3;
109     when others =>
110       IN1 <= (others => '0');
111     end case;
112   end process;
113
114   Mux2: process(R0,R1,R2,R3,SelMux2)
115   begin
116     case SelMux2 is
117     when "00" =>
118       IN2 <= R0;
119     when "01" =>
120       IN2 <= R1;
121     when "10" =>
122       IN2 <= R2;
123     when "11" =>
124       IN2 <= R3;
125     when others =>
126       IN2 <= (others => '0');
127     end case;
128   end process;

```

As shown, the outputs of the MUXes are IN1 and IN2, respectively. These outputs are connected to the first and second inputs of the ALU.

According to the project specification, the ALU in the processor architecture is implemented as follows:



The sensitivity list was defined according to the figure above, and its functionality was implemented as follows:

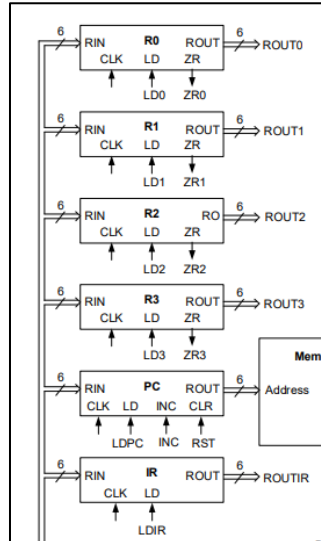
```

145   ALU: process(IN1, IN2, CMD)
146   begin
147     if CMD='0' then
148       ALURes <= IN1 + IN2;
149     elsif CMD='1' then
150       ALURes <= IN1 - IN2;
151     end if;
152   end process;

```

Since there are two ALU operations, CMD is defined as a 1-bit control signal. When CMD is 0, the output is the sum of the two MUX outputs. Otherwise, when CMD is 1, the output is the difference between the two MUX outputs.

According to the project specification, we implement the combinational outputs of the circuit as follows:

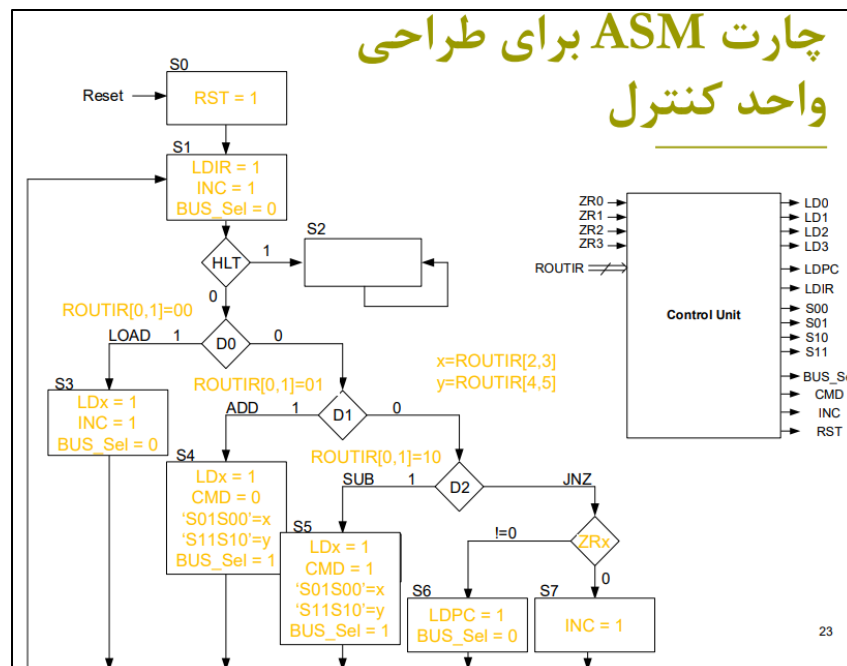


```

131 MData <= Memory(to_integer(unsigned(PC)));
132 ZR0 <= '1' when R0="000000" else '0';
133 ZR1 <= '1' when R1="000000" else '0';
134 ZR2 <= '1' when R2="000000" else '0';
135 ZR3 <= '1' when R3="000000" else '0';
136 PCNext <= DataBUS when LDPC='1' else PC+1 when INC='1' else "000000" when RST='1' else PC;
137 IRNext <= DataBUS when LDIR='1' else IR;
138 R0Next <= DataBUS when LD0='1' else R0;
139 R1Next <= DataBUS when LD1='1' else R1;
140 R2Next <= DataBUS when LD2='1' else R2;
141 R3Next <= DataBUS when LD3='1' else R3;

```

Based on the ASM chart:



The FSM process is implemented using an implicit state machine design:

```

154 process(IR, Z, CurrentState)
155 begin
156   -- Initialize signals
157   CMD <= '0';
158   INC <= '0';
159   RST <= '0';
160   LD0 <= '0';
161   LD1 <= '0';
162   LD2 <= '0';
163   LD3 <= '0';
164   LDPC <= '0';
165   LDIR <= '0';
166   SelMux1 <= "00";
167   SelMux2 <= "00";
168   BUSSel <= '0';
169
170   -- State transitions
171   case CurrentState is
172     when s0 =>
173       RST <= '1';
174       NextState <= s1;
175
176     when s1 =>
177       LDIR <= '1';
178       INC <= '1';
179       BUSSel <= '0';
180       NextState <= HaltCheck;
181
182     when HaltCheck =>
183       if IR = "111111" then
184         NextState <= s2;
185       elsif IR(5 downto 4) = "00" then
186         NextState <= s3;
187       elsif IR(5 downto 4) = "01" then
188         NextState <= s4;
189       elsif IR(5 downto 4) = "10" then
190         NextState <= s5;
191       elsif IR(5 downto 4) = "11" then
192         if Z(index) = '0' then
193           NextState <= s6;
194         else
195           NextState <= s7;
196         end if;
197     end if;

```

```

199   when s2 =>
200     NextState <= s2;
201
202   when s3 =>
203     NextState <= s1;
204     INC <= '1';
205     BUSSel <= '0';
206     case IR(3 downto 2) is
207       when "00" => LD0 <= '1';
208       when "01" => LD1 <= '1';
209       when "10" => LD2 <= '1';
210       when others => LD3 <= '1';
211     end case;

```

```

213   when s4 =>
214     NextState <= s1;
215     CMD <= '0';
216     SelMux1 <= IR(3 downto 2);
217     SelMux2 <= IR(1 downto 0);
218     BUSSel <= '1';
219
220     case IR(3 downto 2) is
221       when "00" => LD0 <= '1';
222       when "01" => LD1 <= '1';
223       when "10" => LD2 <= '1';
224       when others => LD3 <= '1';
225     end case;
226
227   when s5 =>
228     NextState <= s1;
229     CMD <= '1';
230     SelMux1 <= IR(3 downto 2);
231     SelMux2 <= IR(1 downto 0);
232     BUSSel <= '1';
233
234     case IR(3 downto 2) is
235       when "00" => LD0 <= '1';
236       when "01" => LD1 <= '1';
237       when "10" => LD2 <= '1';
238       when others => LD3 <= '1';
239     end case;

```

```

241     when s6 =>
242         nextState <= s1;
243         LDPC <= '1';
244         BUSsel <= '0';
245
246     when s7 =>
247         INC <= '1';
248         nextState <= s1;
249
250 end case;
251 end process;
252
253 end Processor;

```

It should be noted that, to simplify the code for the HaltCheck state, an array Z was used instead of separately using ZR0, ZR1, ZR2, and ZR3.

Implementation of Step 1

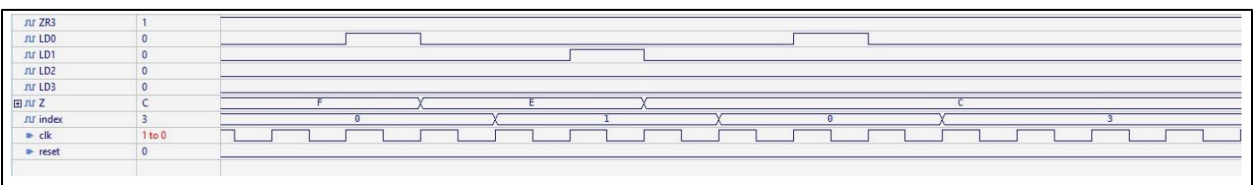
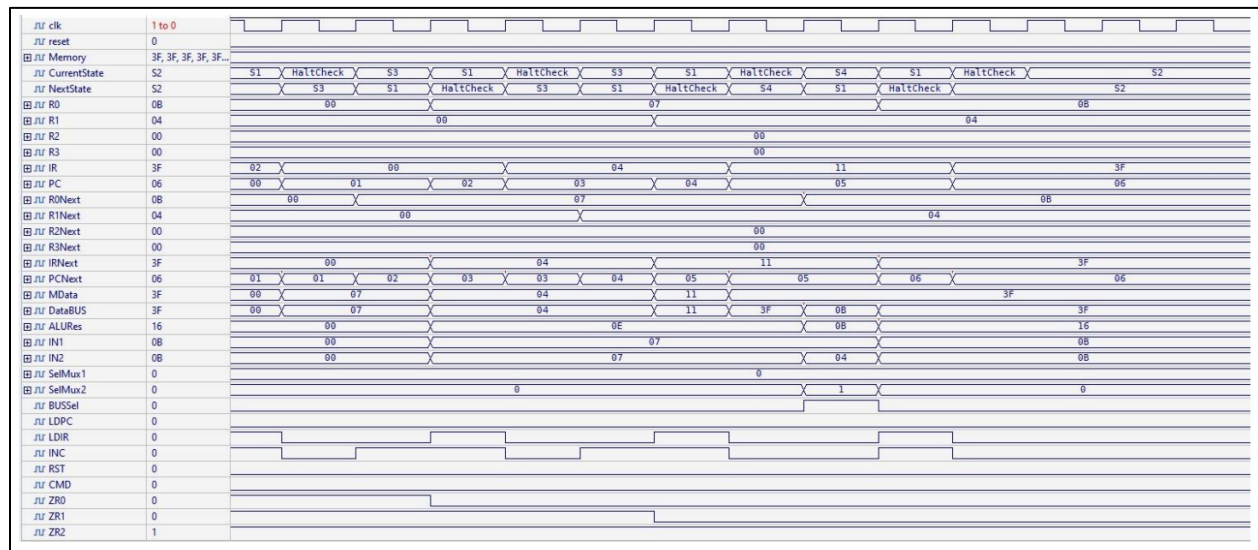
To verify the functionality of Step 1, we write the code specified in the first step into the processor memory:

```

12 type Memory_t is array (63 downto 0) of std_logic_vector(5 downto 0);
13 type State_t is (S0, HaltCheck, S1, S2, S3, S4, S5, S6, S7);
14
15 signal Memory : Memory_t :=
16 (
17     -- PART 1:
18     0 => "000000", -- Load R0,
19     1 => "000111", -- 7
20     2 => "000100", -- Load R1,
21     3 => "000100", -- 4
22     4 => "010001", -- Add, R0, R1
23     others => "111111" -- Halt
24 );
25

```

Note that 111111 is defined as the Halt instruction.



As shown in the result, the final value stored in Register 0 is 0B, which corresponds to 11 in decimal ($4 + 7 = 11$).

Step 2: Software Multiplication

For this step, we used the assembler we developed.

```
LOAD r0, 6
LOAD r1, 8
LOAD r2, 1
LOAD r3, 0
L0:add r3, r0
sub r1, r2
jnz r1, L0
```

In this section, we provide the assembly code to the Python-based assembler, which generates the corresponding binary machine code.

In this section, we import the required library:

```
sembler
2   from typing import Dict, List
3
```

In this section, we search for a substring within a string in a case-insensitive manner.

```
3 usages
@staticmethod
def find_ci(data: str, to_search: str, pos: int = 0) -> bool:
    data = data.lower()
    return to_search in data[pos:]
```

In this section, the string is split using the specified delimiter.

```
3 usages
@staticmethod
def split(input_string: str, delimiter: str) -> List[str]:
    return input_string.split(delimiter)
```

In this section, the number is converted into a 6-bit binary string, with its value clamped between 0 and 63.


```

2 usages
@staticmethod
def to_binary(num: int) -> str:
    num = max(0, min(num, 63))
    return format(num, '06b')

```

Opcode and Register:

```

20  ∨ opcodes: Dict[str, str] = {
21      "load": "00",
22      "add": "01",
23      "sub": "10",
24      "jnz": "11"
25  }
26
27  ∨ registers: Dict[str, str] = {
28      "r0": "00",
29      "r1": "01",
30      "r2": "10",
31      "r3": "11"
32  }
33

```

Assembler Class:

```

1 usage
class Assembler:
    def __init__(self, input_string: str):
        self.m_Input = input_string
        self.m_CurrentLabel = 1
        self.m_LabelAddress = {}

```

input_string: The assembly code provided as a string.

m_CurrentLabel: Keeps track of the current label number (not used in the provided code).

m_LabelAddress: Stores the addresses of labels.

assemble method: Converts the input assembly code into binary machine code.

```

40     def assemble(self) -> str:
41         self.calculate_label_addresses()
42         m_Output = ""
43         for line in self.m_Input.splitlines():
44             line = line.replace(_old: ',', _new: '')
45             instruction = Helpers.split(line, delimiter: ' ')
46             opcode = instruction[0]
47
48             if ':' in opcode:
49                 opcode = Helpers.split(opcode, delimiter: ':')[1]
50
51             m_Output += opcodes[opcode]
52             m_Output += registers[instruction[1]]
53
54             if Helpers.find_ci(opcode, to_search: "load"):
55                 m_Output += "00\n"
56                 m_Output += Helpers.to_binary(int(instruction[2]))
57             elif Helpers.find_ci(opcode, to_search: "jnz"):
58                 m_Output += "00\n"
59                 m_Output += Helpers.to_binary(self.m_LabelAddress[instruction[2]])
60             else:
61                 m_Output += registers[instruction[2]]
62
63             m_Output += "\n"
64
65         m_Output += "111111"
66         return m_Output

```

Reads the input code line by line.

Parses each instruction and converts it into binary using the dictionaries.

Specifically handles LOAD and JNZ instructions by adding the binary representation of immediate values or label addresses.

Appends the 111111 terminator at the end.

get_command_size method:

```

1 usage
def get_command_size(self, command: str) -> int:
    if Helpers.find_ci(command, to_search: "load"):
        return 2
    else:
        return 1

```

This method returns the size of an instruction, which is either 1 or 2 words.

calculate_label_addresses method:

```

1 usage
def calculate_label_addresses(self):
    current_address = 0
    for line in self.m_Input.splitlines():
        if ':' in line:
            self.m_LabelAddress[Helpers.split(line, delimiter: ':')[0]] = current_address
            current_address += self.get_command_size(line)

```

This method calculates the addresses of labels in the assembly code.

The final section reads the input assembly file and prints the assembled binary output.

```

81 ▶ if __name__ == "__main__":
82     input_file = 'input.txt'
83     with open(input_file, 'r') as file:
84         input_string = file.read()
85
86     assembler = Assembler(input_string)
87     print("output:\n" + assembler.assemble())
88

```

The output for our input is as follows:

```

output:
000000
000110
000100
001000
000100
000001
001100
000000
011100
100110
110100
001000
111111

Process finished with exit code 0

```

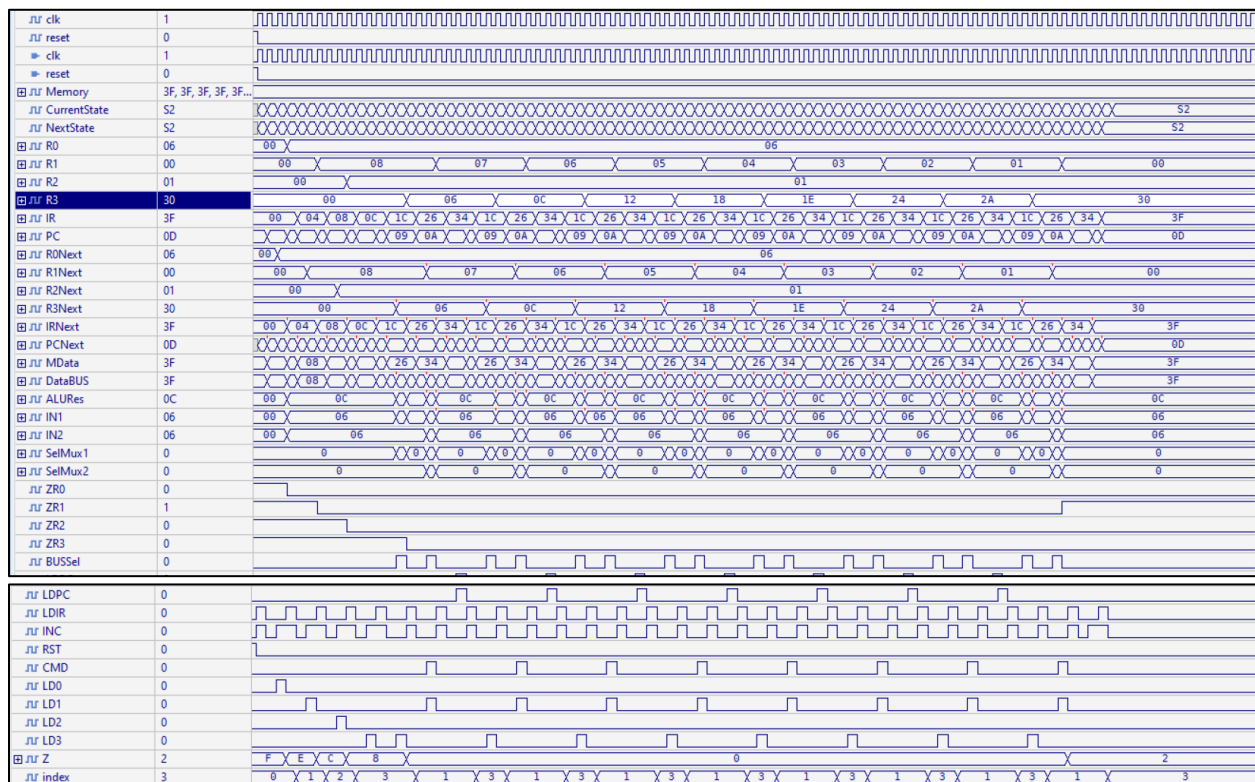
We then write the machine code generated by the assembler into the processor memory:

```

15 signal Memory : Memory_t :=
16 (
17   -- PART 1:
18   --0    => "000000", -- Load R0,
19   -- 1    => "000111", -- 7
20   -- 2    => "000100", -- Load R1,
21   -- 3    => "000100", -- 4
22   --4    => "010001", -- Add, R0, R1
23   --others => "111111" -- Halt
24
25   --PART 2:
26   0 => "000000",
27   1 => "000110",
28   2 => "000100",
29   3 => "001000",
30   4 => "001000",
31   5 => "000001",
32   6 => "001100",
33   7 => "000000",
34   8 => "011100",
35   9 => "100110",
36   10 => "110100",
37   11 => "001000",
38   others => "111111"
39
40 );

```

Implementation of Step 2



As shown, the value 30 in hexadecimal, which is equivalent to 48 in decimal, is stored in register R3.

Step 3: Hardware Multiplication Extension

To add the MULT instruction, we need to extend the ALU. To do this, the ALU control signal must be increased from 1 bit to 2 bits. We also need to increase the width of ALURes so that it can represent the multiplication result.

Therefore, the new ALU is implemented as follows:

```
129     process(IN1, IN2, CMD)
130     begin
131     case CMD is
132         when "00" =>
133             ALURes <= "0000000" & (IN1 + IN2);
134         when "01" =>
135             ALURes <= "0000000" & (IN1 - IN2);
136         when "10" =>
137             ALURes <= IN1 * IN2;
138         when others =>
139             ALURes <= (others => '0');
140     end case;
141 end process;
```

In the code above, there are three valid cases. The others case should not normally occur and would only be reached if an unexpected condition or error occurred. It was included to cover all possible case values, with the ALU output set to all zeros.

Since a new instruction is being added, the opcode width must be increased from 2 bits to 3 bits. The existing opcodes remain unchanged, with a leading 0 added to each of them, while the new MULT opcode is defined as 100. Therefore, the memory word size must also be increased from 6 bits to 7 bits to accommodate the new opcode and instruction format.

Next, a new FSM state is required to execute the multiplication operation. We define a new state called S8 and check for the MULT instruction in the Halt or Not state. If the IR contains the multiplication opcode 100, the FSM transitions to state S8.

```
187         elsif IR(6 downto 4) = "100" then
188             NextState <= s8;
189         else
190             NextState <= s1;
191         end if;
```

The other conditions in this section remain the same as before, with one additional 0 bit prefixed to each opcode.

For the multiplication operation, we have:

```
244     when s8 =>
245         NextState <= s1;
246         CMD <= "10";
247         BUSSel <= '1';
248         SelMux1 <= IR(3 downto 2);
249         SelMux2 <= IR(1 downto 0);
250
251     case IR(3 downto 2) is
252         when "00" => LD0 <= '1';
253         when "01" => LD1 <= '1';
254         when "10" => LD2 <= '1';
255         when others => LD3 <= '1';
256     end case;
```

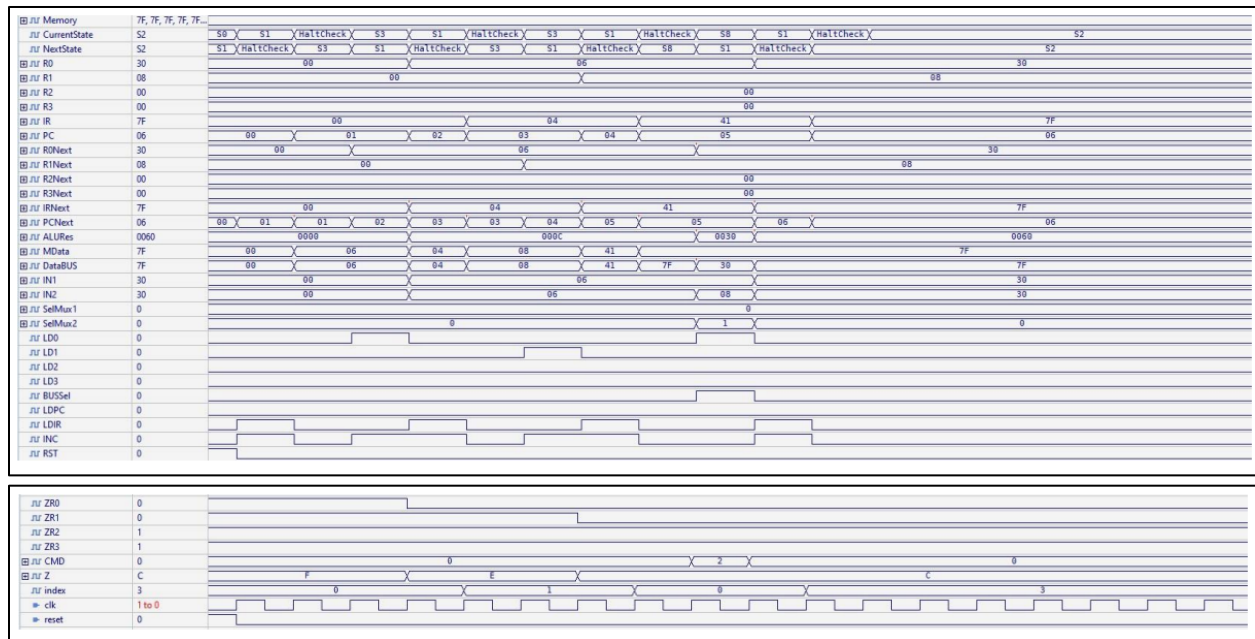
Finally, we write the code for multiplying 6 by 8 into the processor memory:

```

15 signal Memory : Memory_t :=
16 (
17   --PART 3:
18   0 => "00000000", -- Load R0,
19   1 => "0000110", -- 6
20   2 => "0000100", -- Load R1,
21   3 => "0001000", -- 8
22   4 => "1000001", -- Mult, R0, R1
23   others => "1111111" -- Halt
24 );

```

Implementation of Step 3



As shown, the product of 6 and 8 is stored in register R0. The hexadecimal value 30 is equivalent to 48 in decimal, confirming that the processor is functioning correctly.

Python Assembler

The assembly code written for Step 2 was relatively long, so we used the assembler we developed to generate the corresponding binary machine code.

The Python assembler supports labels and automatically calculates their addresses and inserts the corresponding address into JNZ instructions, eliminating the need to hardcode jump addresses. As shown, this assembler functionality was also used in Step 2.